

🛡️ Estrategia de Persistencia de Datos - Multi-App

📊 ANÁLISIS DE RIESGOS ACTUALES

Configuración Actual:

```
volumes:
  postgres_data: # Volume Docker sin especificar ubicación
  static_volume: # Volume Docker sin especificar ubicación
```

Problemas Identificados:

- × **Ubicación incierta:** Docker maneja los volúmenes internamente
- × **Backup complicado:** Acceso difícil a los datos
- × **Migración problemática:** Dependiente de Docker volumes
- × **Sin redundancia:** Un solo punto de fallo
- × **Permisos confusos:** Ownership dentro del contenedor

🚀 ESTRATEGIA DE PERSISTENCIA EMPRESARIAL

1. VOLÚMENES EXPLÍCITOS CON BIND MOUNTS

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_MULTIPLE_DATABASES: ecodisseny, oscar_shop
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      # ✅ PERSISTENCIA EXPLÍCITA
      - /opt/ecodisseny/data/postgres:/var/lib/postgresql/data
      - /opt/ecodisseny/backups/postgres:/var/backups/postgresql
      - /opt/ecodisseny/config/postgres:/etc/postgresql/conf.d
      - /opt/ecodisseny/logs/postgres:/var/log/postgresql
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s
```

```
ecodisseny:
  build: ./ecodisseny
  restart: unless-stopped
  volumes:
    # ✔ DATOS PERSISTENTES EN HOST
    - /opt/ecodisseny/data/media:/app/media
    - /opt/ecodisseny/data/static:/app/staticfiles
    - /opt/ecodisseny/logs/django:/app/logs
    - /opt/ecodisseny/uploads:/app/uploads
  environment:
    - DB_HOST=postgres
    - DB_NAME=ecodisseny
  depends_on:
    postgres:
      condition: service_healthy

oscar_shop:
  build: ./oscar_shop
  restart: unless-stopped
  volumes:
    # ✔ DATOS SEPARADOS POR APP
    - /opt/ecodisseny/data/oscar/media:/app/media
    - /opt/ecodisseny/data/oscar/static:/app/staticfiles
    - /opt/ecodisseny/logs/oscar:/app/logs
  environment:
    - DB_HOST=postgres
    - DB_NAME=oscar_shop
  depends_on:
    postgres:
      condition: service_healthy

redis:
  image: redis:7-alpine
  restart: unless-stopped
  volumes:
    - /opt/ecodisseny/data/redis:/data
  command: redis-server --appendonly yes --maxmemory 512mb

pgadmin:
  image: dpape/pgadmin4:latest
  restart: unless-stopped
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@ecodisseny.com
    PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD}
  volumes:
    - /opt/ecodisseny/data/pgadmin:/var/lib/pgadmin
  ports:
    - "127.0.0.1:8080:80"
```

2. ESTRUCTURA DE DIRECTORIOS EN HOST

```

/opt/ecodisseny/
├── data/
│   ├── postgres/      # Base de datos PostgreSQL
│   ├── redis/         # Cache Redis
│   ├── media/         # Archivos subidos Ecodisseny
│   ├── static/        # Archivos estáticos Ecodisseny
│   ├── oscar/
│   │   ├── media/     # Archivos Oscar Shop
│   │   └── static/    # Estáticos Oscar Shop
│   └── pgadmin/       # Configuración pgAdmin
├── backups/
│   ├── postgres/     # Backups automáticos DB
│   ├── daily/        # Backups diarios
│   ├── weekly/       # Backups semanales
│   └── monthly/      # Backups mensuales
├── config/
│   ├── postgres/     # Configuración PostgreSQL
│   ├── nginx/        # Configuración Nginx
│   └── ssl/          # Certificados SSL
└── logs/
    ├── postgres/     # Logs PostgreSQL
    ├── django/       # Logs Django Ecodisseny
    ├── oscar/        # Logs Oscar Shop
    └── nginx/        # Logs Nginx

```

SISTEMA DE BACKUP AUTOMATIZADO

Script de Backup Completo:

```

#!/bin/bash
# /opt/ecodisseny/scripts/backup_completo.sh

set -e

BACKUP_DIR="/opt/ecodisseny/backups"
DATE=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=30

echo "🔄 Iniciando backup completo - $DATE"

# 1. BACKUP POSTGRESQL
echo "📁 Backup PostgreSQL..."
docker exec ecodisseny-postgres-1 pg_dumpall -U postgres > \
    "$BACKUP_DIR/postgres/full_backup_$DATE.sql"

# 2. BACKUP POR BASE DE DATOS
echo "🔧 Backup individual databases..."
docker exec ecodisseny-postgres-1 pg_dump -U postgres ecodisseny > \
    "$BACKUP_DIR/postgres/ecodisseny_$DATE.sql"

```

```

docker exec ecodisseny-postgres-1 pg_dump -U postgres oscar_shop > \
"$BACKUP_DIR/postgres/oscar_shop_$(date +%Y%m%d).sql"

# 3. BACKUP ARCHIVOS MEDIA
echo "📁 Backup archivos media..."
tar -czf "$BACKUP_DIR/daily/media_ecodisseny_$(date +%Y%m%d).tar.gz"
/opt/ecodisseny/data/media/
tar -czf "$BACKUP_DIR/daily/media_oscar_$(date +%Y%m%d).tar.gz"
/opt/ecodisseny/data/oscar/media/

# 4. BACKUP CONFIGURACIÓN
echo "⚙️ Backup configuración..."
tar -czf "$BACKUP_DIR/daily/config_$(date +%Y%m%d).tar.gz" /opt/ecodisseny/config/

# 5. LIMPIEZA DE BACKUPS ANTIGUOS
echo "🧹 Limpiando backups antiguos..."
find "$BACKUP_DIR/daily" -name "*.tar.gz" -mtime +$RETENTION_DAYS -delete
find "$BACKUP_DIR/postgres" -name "*.sql" -mtime +$RETENTION_DAYS -delete

# 6. VERIFICACIÓN DE INTEGRIDAD
echo "✅ Verificando integridad..."
for backup in "$BACKUP_DIR/postgres"/*_$(date +%Y%m%d).sql; do
    if [ -f "$backup" ] && [ -s "$backup" ]; then
        echo "✅ $backup - OK"
    else
        echo "❌ $backup - FALLO"
        exit 1
    fi
done

echo "🎉 Backup completo finalizado - $(date +%Y%m%d)"

```

Cron para Backups Automáticos:

```

# Crontab para backups automáticos
# crontab -e

# Backup completo diario a las 2:00 AM
0 2 * * * /opt/ecodisseny/scripts/backup_completo.sh >>
/opt/ecodisseny/logs/backup.log 2>&1

# Backup incremental cada 6 horas
0 */6 * * * /opt/ecodisseny/scripts/backup_incremental.sh >>
/opt/ecodisseny/logs/backup.log 2>&1

# Backup semanal (domingos a las 1:00 AM)
0 1 * * 0 /opt/ecodisseny/scripts/backup_semanal.sh >>
/opt/ecodisseny/logs/backup.log 2>&1

```

SCRIPTS DE MANTENIMIENTO

1. Script de Inicialización:

```
#!/bin/bash
# /opt/ecodisseny/scripts/init_persistencia.sh

echo "🚀 Inicializando estructura de persistencia..."

# Crear directorios
sudo mkdir -p
/opt/ecodisseny/{data/{postgres,redis,media,static,oscar/{media,static},pgadmin},backups/{postgres,daily,weekly,monthly},config/{postgres,nginx,ssl},logs/{postgres,django,oscar,nginx}}

# Establecer permisos
sudo chown -R 999:999 /opt/ecodisseny/data/postgres
sudo chown -R 999:999 /opt/ecodisseny/data/redis
sudo chown -R www-data:www-data /opt/ecodisseny/data/media
sudo chown -R www-data:www-data /opt/ecodisseny/data/static
sudo chown -R 5050:5050 /opt/ecodisseny/data/pgadmin

# Crear archivos de configuración
sudo touch
/opt/ecodisseny/logs/{postgres/postgresql.log,django/django.log,oscar/oscar.log,nginx/access.log,nginx/error.log}

echo "✅ Estructura de persistencia creada"
```

2. Script de Restauración:

```
#!/bin/bash
# /opt/ecodisseny/scripts/restore_backup.sh

if [ -z "$1" ]; then
    echo "Uso: $0 <archivo_backup.sql>"
    exit 1
fi

BACKUP_FILE="$1"
TEMP_DB="restore_temp_$(date +%s)"

echo "🔄 Restaurando desde: $BACKUP_FILE"

# 1. Crear base de datos temporal
docker exec ecodisseny-postgres-1 createdb -U postgres "$TEMP_DB"

# 2. Restaurar en DB temporal
docker exec -i ecodisseny-postgres-1 psql -U postgres "$TEMP_DB" <
"$BACKUP_FILE"
```

```
# 3. Verificar integridad
if docker exec ecodisseny-postgres-1 psql -U postgres "$TEMP_DB" -c "\dt" >
/dev/null 2>&1; then
    echo "✓ Backup verificado correctamente"

# 4. Confirmar restauración
read -p "¿Confirmar restauración? (y/N): " confirm
if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
    echo "🔄 Restaurando base de datos principal..."
    docker-compose down
    docker exec ecodisseny-postgres-1 dropdb -U postgres ecodisseny
    docker exec ecodisseny-postgres-1 createdb -U postgres ecodisseny
    docker exec -i ecodisseny-postgres-1 psql -U postgres ecodisseny <
"$BACKUP_FILE"
    docker-compose up -d
    echo "✓ Restauración completada"
fi
else
    echo "✗ Error en la verificación del backup"
fi

# Limpiar DB temporal
docker exec ecodisseny-postgres-1 dropdb -U postgres "$TEMP_DB"
```

📊 MONITOREO Y ALERTAS

Script de Monitoreo:

```
#!/bin/bash
# /opt/ecodisseny/scripts/monitor_persistencia.sh

ALERT_EMAIL="admin@ecodisseny.com"

# 1. Verificar espacio en disco
DISK_USAGE=$(df /opt/ecodisseny | awk 'NR==2 {print $5}' | sed 's/%//')
if [ "$DISK_USAGE" -gt 85 ]; then
    echo "⚠ ALERTA: Uso de disco al ${DISK_USAGE}%" | mail -s "Alerta
Disco" "$ALERT_EMAIL"
fi

# 2. Verificar estado de contenedores
for container in postgres redis ecodisseny oscar_shop; do
    if ! docker ps | grep -q "$container"; then
        echo "✗ ALERTA: Contenedor $container no está ejecutándose" | mail
-s "Alerta Contenedor" "$ALERT_EMAIL"
    fi
done

# 3. Verificar tamaño de bases de datos
```

```
DB_SIZE=$(docker exec ecodisseny-postgres-1 psql -U postgres -c "SELECT
pg_size_pretty(pg_database_size('ecodisseny'));" | grep -oP '\d+\s\w+')
echo "📊 Tamaño BD Ecodisseny: $DB_SIZE"

# 4. Verificar última copia de seguridad
LAST_BACKUP=$(ls -t /opt/ecodisseny/backups/postgres/*.sql | head -1)
BACKUP_AGE=$((($(date +%s) - $(stat -c %Y "$LAST_BACKUP")) / 3600))
if [ "$BACKUP_AGE" -gt 25 ]; then
    echo "⚠️ ALERTA: Último backup hace ${BACKUP_AGE} horas" | mail -s
"Alerta Backup" "$ALERT_EMAIL"
fi
```



SEGURIDAD DE DATOS

1. Encriptación en Reposo:

```
# Configuración PostgreSQL con encriptación
postgres:
  environment:
    POSTGRES_INITDB_ARGS: "--auth-host=md5 --auth-local=md5"
  volumes:
    -
      /opt/ecodisseny/config/postgres/postgresql.conf:/etc/postgresql/postgresql.
      conf
```

Archivo de configuración PostgreSQL:

```
# /opt/ecodisseny/config/postgres/postgresql.conf

# Seguridad
ssl = on
ssl_cert_file = '/etc/ssl/certs/postgres.crt'
ssl_key_file = '/etc/ssl/private/postgres.key'

# Logging
log_statement = 'mod'
log_min_duration_statement = 1000
log_connections = on
log_disconnections = on

# Backups
archive_mode = on
archive_command = 'cp %p /var/backups/postgresql/archive/%f'
```

2. Backup Cifrado:

```
# Backup con cifrado GPG
docker exec ecodisseny-postgres-1 pg_dumpall -U postgres | \
  gpg --cipher-algo AES256 --compress-algo 1 --symmetric --output \
    "/opt/ecodisseny/backups/postgres/encrypted_backup_$(date
    +%Y%m%d).sql.gpg"
```

✓ CHECKLIST DE IMPLEMENTACIÓN

Fase 1: Preparación

- ☐ Crear estructura de directorios
- ☐ Configurar permisos correctos
- ☐ Instalar scripts de backup
- ☐ Configurar cron jobs

Fase 2: Migración

- ☐ Backup de datos actuales
- ☐ Modificar docker-compose.yml
- ☐ Migrar volúmenes Docker a bind mounts
- ☐ Verificar funcionamiento

Fase 3: Automatización

- ☐ Configurar backups automáticos
- ☐ Implementar monitoreo
- ☐ Configurar alertas
- ☐ Documentar procedimientos

Fase 4: Validación

- ☐ Probar restauración completa
- ☐ Verificar integridad de datos
- ☐ Validar procedimientos de emergencia
- ☐ Entrenar al equipo

🎯 BENEFICIOS DE ESTA ESTRATEGIA

✓ Garantías de Persistencia:

1. **Ubicación conocida:** Todos los datos en `/opt/ecodisseny/`
2. **Backup sistemático:** Automático y verificado
3. **Restauración rápida:** Procedimientos documentados
4. **Escalabilidad:** Fácil añadir nuevas apps
5. **Migración simple:** Solo copiar `/opt/ecodisseny/`

📊 Métricas de Seguridad:

- **RTO (Recovery Time Objective):** < 15 minutos
- **RPO (Recovery Point Objective):** < 6 horas
- **Disponibilidad:** 99.9%
- **Integridad:** Verificación automática

¿Quieres que implemente esta estrategia de persistencia completa?